

Instrucciones para ejecutar y replicar el experimento

Programación de la Producción — Máster UPV

Rafael Macian · Patricia Rodrigo · José Ramón Mena · Víctor Rodríguez

Este documento describe paso a paso cómo ejecutar los dos archivos Python del experimento, qué debe aparecer por pantalla, qué se escribe en el Excel, y cómo interpretar los resultados. Cualquier persona que siga estas instrucciones debe obtener exactamente los mismos números que los publicados en el trabajo.

Índice

1. Visión general del experimento	3
2. Requisitos previos	3
2.1. Dependencias Python	3
2.2. Archivos necesarios	3
3. Ajuste de rutas	4
4. Paso 1: Ejecutar 01_generar_datos.py	4
4.1. Qué hace este archivo	4
4.2. Cómo se genera cada parámetro	4
4.3. Qué debe aparecer por pantalla	6
4.4. Qué se escribe en el Excel	6
5. Paso 2: Ejecutar 02_extensiones.py	7
5.1. Progreso por pantalla	7
5.2. Resultado final esperado	7
6. Ejecución parcial y guardado de progreso	7
6.1. Ejecutar una sola extensión con Run Above	7
6.2. Checkpoint en JSON	8
7. Reproducibilidad: por qué siempre sale lo mismo	8
7.1. Semillas fijas	8
7.2. Parada por iteraciones, no por tiempo	8
7.3. Redondeos	8
8. Preguntas que nos surgieron	9
A. Tabla completa de parámetros	10

1. Visión general del experimento

El experimento se divide en dos archivos Python que deben ejecutarse **en orden** y el Excel base con los tiempos de proceso de cada trabajo por máquina. La Figura 1 muestra el flujo completo.

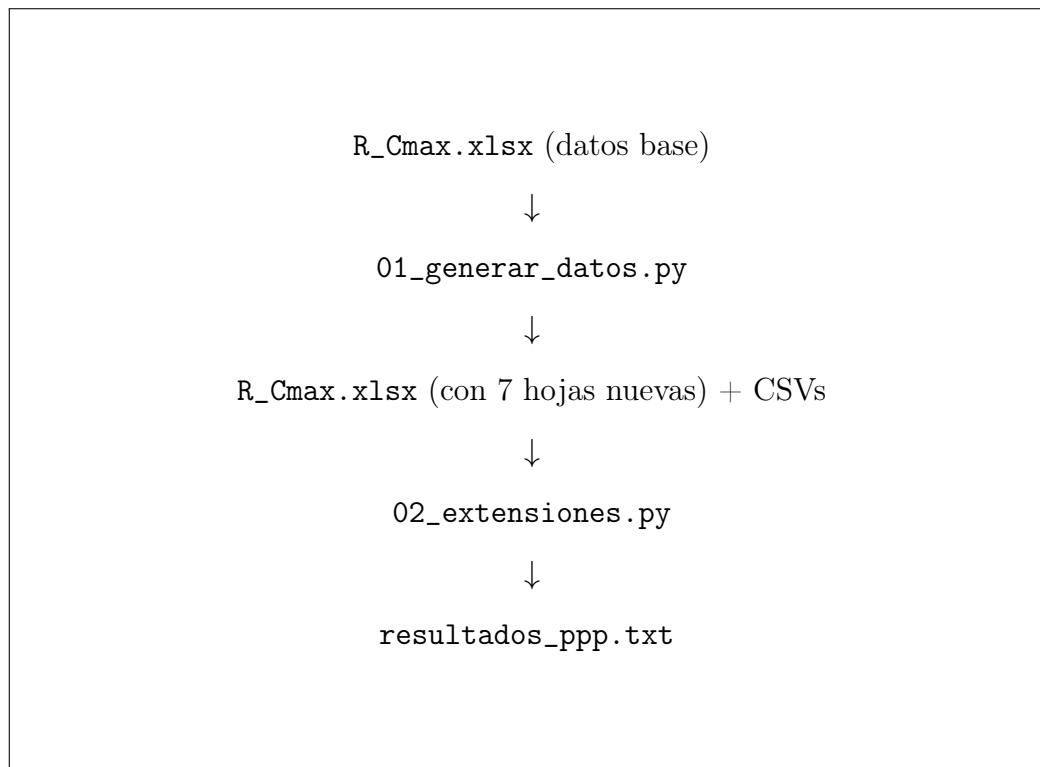


Figura 1: Flujo completo del experimento

2. Requisitos previos

2.1. Dependencias Python

Ambos archivos gestionan automáticamente la instalación de las librerías necesarias al ejecutarse. No es necesario instalar nada manualmente. Las librerías utilizadas son `openpyxl` (lectura y escritura del Excel), `numpy` (cálculo de medias) y `ortools` (solver CP-SAT).

2.2. Archivos necesarios

- `R_Cmax.xlsx` — Excel base con los tiempos de proceso. Los datos deben estar en la hoja activa empezando en la celda **B15**, con 50 filas (trabajos) y 5 columnas (máquinas). Si en tu Excel la tabla estuviera en una posición diferente, basta con ajustar las variables `row_inicio` y `col_inicio` al inicio de la función `leer_tiempos_proceso()` en `01_generar_datos.py`.
- `01_generar_datos.py`
- `02_extensiones.py`

3. Ajuste de rutas

Ambos archivos tienen una sección marcada con bloques `###` al principio donde se indican las rutas a modificar. Son las **únicas líneas que hay que cambiar**.

```
1 RUTA_EXCEL = r"C:\Users\joser\...\R_Cmax.xlsx"
2 # Sustituir por la ruta completa al Excel en tu ordenador
```

Listing 1: Rutas a ajustar en 01_generar_datos.py (línea 19)

```
1 RUTA_EXCEL = r"C:\Users\joser\...\R_Cmax.xlsx"
2 # Mismo Excel que antes
3
4 RUTA_TXT = r"C:\Users\joser\...\resultados_ppp.txt"
5 # Ruta donde se guardar el archivo de resultados
```

Listing 2: Rutas a ajustar en 02_extensiones.py (líneas 21 y 27)

4. Paso 1: Ejecutar 01_generar_datos.py

Paso 1

Ejecutar desde la terminal:

```
1 python 01_generar_datos.py
```

O abrir el archivo en VS Code y ejecutarlo completo.

4.1. Qué hace este archivo

Lee la matriz de tiempos de proceso del Excel y genera con semillas fijas los seis parámetros adicionales necesarios para las extensiones. Los escribe como nuevas hojas en el mismo Excel y como CSVs en la carpeta `datos_extensiones`.

4.2. Cómo se genera cada parámetro

Release dates r_j — semilla 123

Los 50 trabajos se dividen en tres grupos según cuándo están disponibles. El 40 % (20 trabajos) recibe un r_j entre 0 y 5, representando trabajos disponibles casi desde el inicio. El 35 % (17 trabajos) recibe un r_j entre 6 y 15, disponibles a mitad del horizonte. El 25 % restante (13 trabajos) recibe un r_j entre 16 y 30, los más tardíos. Una vez generados los tres grupos se mezclan aleatoriamente para que no queden agrupados por bloques en el Excel.

Setups s_{ijk} — semilla 42

Para cada máquina i y cada par de trabajos distintos $j \neq k$ se genera un tiempo de preparación aleatorio entre 1 y el 20 % de la media global de tiempos de proceso $\bar{p}$. La diagonal vale siempre 0 porque preparar una máquina para el mismo trabajo que acaba de procesar no tiene coste. El límite del 20 % de $\bar{p}$ garantiza que los setups son relevantes pero no dominantes respecto a los tiempos de proceso. Si ese 20 % resultara menor que 1, el código fuerza un mínimo de 1 mediante `max(1, int(0.20 * p_media))`.

Due dates d_j — semilla 43

La fecha de entrega de cada trabajo se calcula como:

$$d_j = r_j + \alpha_j \cdot \bar{p}_j$$

donde $\bar{p}_j$ es la media de los tiempos de proceso del trabajo j sobre las 5 máquinas, y α_j es un factor de holgura aleatorio entre 1,2 y 1,8. Con α bajo el plazo es ajustado y es probable que haya tardanza; con α alto hay margen suficiente. El resultado se redondea al entero más cercano. Es posible que para algún trabajo la máquina asignada tenga un tiempo de proceso mayor que la media, haciendo que ese trabajo sea inevitablemente tardío. Esto es intencionado: en la Extensión 5 el objetivo es minimizar el coste acumulado de la tardanza, no eliminarla.

Pesos w_j — semilla 44

Cada trabajo recibe un peso entero aleatorio entre 1 y 5 que representa su prioridad. Un peso de 1 corresponde al cliente menos crítico y un peso de 5 al más crítico. Se utilizan en la Extensión 5 para ponderar la tardanza: un retraso de 10 unidades en un trabajo con $w_j = 5$ penaliza 50 en el objetivo, mientras que el mismo retraso en un trabajo con $w_j = 1$ solo penaliza 10.

Precedencias $\mathcal{P}$ — semilla 45

Se generan 25 pares (j, k) que significan que el trabajo j debe finalizar antes de que el trabajo k pueda comenzar. Para evitar ciclos que harían el problema infactible, solo se consideran pares donde $j < k$ en índice. Así, por construcción, nunca puede existir un bucle de dependencias: un trabajo de índice mayor nunca puede preceder a uno de índice menor. De los $N(N-1)/2 = 1225$ pares posibles se seleccionan aleatoriamente 25, equivalente al 2 % del total.

Recursos h_j y H — semilla 46

Cada trabajo recibe un número de operarios necesarios h_j aleatorio entre 1 y 3. La capacidad total H se fija como:

$$H = \left\lceil 0,6 \cdot \frac{\sum_{j=1}^N h_j}{M} \right\rceil$$

El factor 0,6 es deliberado: con el 100 % siempre habría personal suficiente y la restricción nunca sería activa. Con el 60 % se crea una restricción real que obliga a retrasar algunos trabajos por falta de personal, pero sin hacer el problema irresoluble. El valor mínimo de H está protegido a 2 para evitar casos degenerados.

4.3. Qué debe aparecer por pantalla

Resultado esperado

GENERANDO PARÁMETROS

Resumen:

```
p_media                = 49.07
setup_max (M1, j!=k)   = 9
r[:6]                  = [0, 13, 1, 24, 20, 2]
d[:6]                  = [...]
w[:6]                  = [...]
Nº precedencias        = 25
h[:6]                  = [...]
H (capacidad personal) = ...
```

Escribiendo en Excel y CSVs...

CSV → .../release_dates.csv

CSV → .../setups_M1.csv

...

Excel guardado → .../R_Cmax.xlsx

Generación completada. Ya puedes ejecutar 02_extensiones.py

4.4. Qué se escribe en el Excel

Tras la ejecución, el archivo R_Cmax.xlsx tendrá las siguientes hojas adicionales:

Cuadro 1: Hojas nuevas escritas en el Excel

Hoja	Contenido	Usada en
Release_dates	50 filas: job, r_j	Ext. 1–5
Setups_M1	Matriz 50×50: s_{1jk}	Ext. 2–5
Setups_M2	Matriz 50×50: s_{2jk}	Ext. 2–5
Setups_M3	Matriz 50×50: s_{3jk}	Ext. 2–5
Setups_M4	Matriz 50×50: s_{4jk}	Ext. 2–5
Setups_M5	Matriz 50×50: s_{5jk}	Ext. 2–5
Due_dates	50 filas: job, d_j	Ext. 5
Pesos	50 filas: job, w_j	Ext. 5
Precedencias	25 filas: j_antes , $k_despues$	Ext. 3–5
Recursos	50 filas: job, $h_j + H$ en E1	Ext. 4–5

5. Paso 2: Ejecutar 02_extensiones.py

Paso 2

Ejecutar desde la terminal:

```
1 python 02_extensiones.py
```

Este archivo tarda aproximadamente **2–3 horas** en completarse con los parámetros definidos.

5.1. Progreso por pantalla

El archivo imprime el avance de cada extensión a medida que termina. El orden es siempre: CP-SAT primero (el más lento), Genético, IG.

Resultado esperado

```
>>> Ejecutando EXTENSION 0: R || Cmax ...

>>> Ejecutando EXTENSION 1: R | rj | Cmax ...

...
```

5.2. Resultado final esperado

Al terminar, imprime los resultados de las 6 extensiones y guarda el archivo `resultados_ppp.txt`. La Tabla 2 muestra los valores que deben obtenerse.

Cuadro 2: Resumen de resultados esperados (valores definitivos)

Extensión	Objetivo	IG	Genético	CP-SAT
0 — $R \parallel C_{\text{máx}}$	$C_{\text{máx}}$	168	170	200
1 — $R \mid r_j \mid C_{\text{máx}}$	$C_{\text{máx}}$	169	171	194
2 — $R \mid r_j, s_{ijk} \mid C_{\text{máx}}$	$C_{\text{máx}}$	195	209	670
3 — $R \mid r_j, s_{ijk}, \text{prec} \mid C_{\text{máx}}$	$C_{\text{máx}}$	197	219	230
4 — $R \mid r_j, s_{ijk}, \text{prec}, \text{res} \mid C_{\text{máx}}$	$C_{\text{máx}}$	199	228	263
5 — $R \mid r_j, s_{ijk}, \text{prec}, \text{res}, d_j \mid \sum w_j T_j$	$\sum w_j T_j$	3060	4426	7361

6. Ejecución parcial y guardado de progreso

6.1. Ejecutar una sola extensión con Run Above

El archivo `02_extensiones.py` está dividido en celdas marcadas con `%%`. En editores como VS Code o Spyder es posible ejecutar únicamente la extensión deseada situando el cursor en su celda y usando **Run Above** seguido de **Run Cell**. El *Run Above* es imprescindible porque las funciones y los datos se definen en las celdas superiores.

6.2. Checkpoint en JSON

Dado que la ejecución completa supera las 2 horas, se recomienda implementar un sistema de checkpoint que guarde en un archivo `.json` el resultado de cada extensión en cuanto termina. Si la ejecución se interrumpe, al relanzar el programa se cargan automáticamente los resultados ya guardados y se continúa desde donde se dejó sin repetir las extensiones completadas. El formato JSON es idóneo para este propósito por ser ligero, legible y compatible directamente con los diccionarios de resultados que devuelven las funciones del código.

7. Reproducibilidad: por qué siempre sale lo mismo

7.1. Semillas fijas

Todos los elementos aleatorios del experimento tienen semilla fija. La Tabla 3 recoge cada semilla y su función.

Cuadro 3: Semillas fijas del experimento

Elemento	Función	Semilla
Release dates	Genera los r_j	123
Setups	Genera las matrices s_{ijk}	42
Due dates	Genera los d_j	43
Pesos	Genera los w_j	44
Precedencias	Selecciona los 25 pares	45
Recursos	Genera los h_j	46
Algoritmo Genético	Controla toda la aleatoriedad del AG	9
Iterated Greedy	Controla toda la aleatoriedad del IG	42
CP-SAT	Semilla interna del solver	123

7.2. Parada por iteraciones, no por tiempo

El Genético y el IG paran cuando se alcanza un número máximo de iteraciones o cuando no hay mejora durante un número fijo de iteraciones consecutivas. Esto es fundamental: si la parada fuera por tiempo, un ordenador más rápido haría más iteraciones y daría un resultado diferente. Con parada por iteraciones, el resultado es **idéntico en cualquier ordenador**.

Cuadro 4: Criterios de parada de las metaheurísticas

Criterio	Genético	IG
Máximo de iteraciones	3000 generaciones	800 iteraciones
Paciencia sin mejora	350 generaciones	200 iteraciones

7.3. Redondeos

Los tiempos de proceso se leen del Excel como enteros. Los due dates se generan como enteros (se aplica `int(round(...))` al calcularlos). Los setups son enteros por

construcción. El CP-SAT trabaja con todos los valores escalados por 100 para operar en enteros y evitar errores de coma flotante, y al devolver el resultado divide por 100.

8. Preguntas que nos surgieron

¿Qué pasa si ejecuto 01_generar_datos.py más de una vez? Los datos se sobrescriben pero son idénticos. No hay ningún riesgo.

¿Puede ocurrir que el límite superior de los setups sea menor que 1? No. El código usa `max(1, int(0.20 * p_media))` para garantizar que el límite superior siempre sea al menos 1.

¿Es posible que un trabajo sea inevitablemente tardío en la Extensión 5? Sí, y es intencionado. La fecha de entrega se calcula a partir de la media de los tiempos de proceso, pero el trabajo se ejecuta en una sola máquina que puede tener un tiempo mayor que esa media. El objetivo de la Extensión 5 es minimizar el coste acumulado de la tardanza, no eliminarla completamente.

¿Por qué el CP-SAT devuelve 670 en la Extensión 2 siendo un método exacto? Porque exacto no significa instantáneo. Al incorporar setups, el modelo necesita variables binarias Y_{ijk} y Z_{ijk} para cada par de trabajos en cada máquina, lo que genera del orden de $M \cdot N^2/2 \approx 6125$ variables binarias adicionales. El espacio de búsqueda crece de forma tan abrupta que en 1000 segundos el solver apenas ha explorado una fracción mínima, devolviendo la primera solución factible que ha encontrado aunque no sea de la más alta calidad.

¿Puedo cambiar el número de trabajos o máquinas para reproducir el experimento con otros datos?

Sí. Basta con modificar las variables `N` y `M` al inicio de ambos archivos Python. Todos los algoritmos y generadores de datos se adaptan automáticamente al nuevo tamaño sin necesidad de tocar ninguna otra línea. Eso sí, hay que volver a ejecutar `01_generar_datos.py` para regenerar los parámetros con las nuevas dimensiones.

A. Tabla completa de parámetros

Cuadro 5: Todos los parámetros del experimento

Parámetro	Valor	Descripción
N	50	Número de trabajos
M	5	Número de máquinas
<i>Generación de datos</i>		
Semilla releases	123	
Semilla setups	42	
Semilla due dates	43	
Semilla pesos	44	
Semilla precedencias	45	
Semilla recursos	46	
$s_{\text{máx}}$	$[0,2 \cdot \bar{p}]$	Límite setups
α_j	$U(1,2; 1,8)$	Holgura due dates
n_p	25	Pares de precedencia
H	$[0,6 \cdot \sum h_j / M]$	Capacidad personal
<i>CP-SAT</i>		
Tiempo límite	1000 s	
Hilos	1	
Escala entera	100	
Semilla	123	
<i>Algoritmo Genético</i>		
Tamaño población	160	
Prob. cruce	0,92	
Prob. mutación swap	0,30	
Prob. mutación inversión	0,22	
Elitismo	4	
Máx. generaciones	3000	
Paciencia	350	
Semilla	9	
<i>Iterated Greedy</i>		
Trabajos destruidos d	5	10 % de N
Máx. iteraciones	800	
Paciencia	200	
Semilla	42	